CONTENTS

Where the Error Goes: A Measured Ceiling for Seeded-Basis Weight Compression, and What the Failure Taught Us About Proxies

SeedLM+O authors, 2026

Every number below is measured.

Abstract

We test whether a weight format built from a pseudorandom basis regenerated from a 16-bit LFSR seed, plus an exact outlier side-channel, can compete with production K-quantization at approximately 3.5 bits per weight. It cannot. On Qwen3-1.7B, measured against same-stack `Q3_K_M` (imatrix) at 0.205 mean KL @ 3.85 bpw, our best configuration reaches 0.436 @ 3.48 bpw, a 2.1x quality gap that survived every optimization attempted. Both preregistered gates fail.

The interesting content is in *how* it failed. (i) Changing only the fit objective from plain L2 to an activation-weighted L2 improved KL **2.6x at byte-identical cost** (1.121 -> 0.436 at a matched 65,535-seed budget) while plain relative weight error moved the *wrong way*, 0.2192 -> 0.2534: the binding constraint was the residual's *structure*. (The raw 4.17x distance from the first plain-L2 row, 1.818 -> 0.436, crosses two variables and decomposes into a 1.62x seed-budget factor and this 2.6x objective factor.) (ii) A strictly tighter minimizer of that same weighted objective then improved the objective 1.7% and degraded KL 11%, with an entire family of fits displaced +8.8% off the reference proxy-to-KL curve; yet the same exact-rounding search run under plain L2 *improved* KL 16%, so the displacement traces to the weighted metric itself. (iii) Our calibration set was our evaluation set: two disjoint held-out 128-paragraph calibration draws land at KL 0.551 and 0.656, a 0.106 spread that is the noise floor of any weighted comparison, and the leaked 0.436 sits below that entire band, an effect size of 0.12 to 0.22 KL. (iv) The method degrades with scale (0.6B to 1.7B, KL 0.759 -> 1.121 at matched seeds) while weight-space error stays scale-invariant (0.2180 -> 0.2192), inverting the usual expectation that larger models are more compressible.

We release the harness, every result row, and the negative findings.

---

1. Introduction

1.1 THE IDEA

A quantized weight tensor spends nearly all of its bits describing the *bulk* of its values: the dense, unremarkable mass that carries little individual information but collectively determines behaviour. Seeded compression proposes that the bulk does not need to be described at all. If a block of weights can be approximated as a linear combination of `P` pseudorandom basis vectors generated deterministically from a 16-bit LFSR seed, the storage cost collapses to: the seed, a shared exponent, and `P` low-precision coefficients. The decoder regenerates the basis; the bits never travel.

The obvious weakness is that a random basis fits *badly*. The obvious patch, and the "+O" of SeedLM+O, is an exact outlier side-channel: spend a small, tunable per-tensor budget storing the values the seeded fit got most wrong, at full precision, indexed.

The bit accounting is what makes this seductive. At twelve weights per block and four coefficients, the bulk costs 36 bits per block (3.00 bits per weight) before any individual value has been described at all, and a 1% exact side-channel adds 0.48. That lands at 3.48 bpw, inside the band `Q3_K_M` occupies, with a format that spends *nothing* on the mass of the tensor. Every bit the bulk does not consume is a bit available to the entries that actually matter, and the outlier channel is a direct, tunable way to spend it there.

The niche also looked empty. Seeded pseudorandom bases and exact outlier side-channels each appear in the literature; we could not find the combination evaluated in the sub-4-bpw regime against measured K-quant comparators. An unoccupied niche with a clean arithmetic argument behind it is worth one cheap probe.

The probe passed convincingly. On Qwen3-0.6B at `c8p3`, the AWQ outlier channel cut mean KL **38.75%** below the seed-only fit and closed **88.9%** of the remaining gap to RTN4 (0.184999 at 4.4800 bpw against RTN4's 0.170426 at 4.5000), a STRONG verdict under the preregistered Phase 2 rule. That is the result that motivated the scale-up.

Scaling to Qwen3-1.7B inverted it. The method got *worse* with model size while weight-space error did not move (§8); the fit objective, not the bit budget, turned out to be the binding constraint (§5); and pushing harder on the objective that had just delivered a 2.6x win made the model worse (§6). The verdict against real comparators is a measured 2.1x quality gap that nothing in the optimization ladder closed.

So this is a methods paper about a format that did not work. The four findings in §1.3 are the contribution; the gate failure in §4 is the measurement that makes them worth reporting. The question the format was built to answer, whether the arithmetic works out against a real, deployed alternative, has an answer, and the useful content is in the route to it.

1.2 MEASUREMENT RULES

Compression papers fail in a characteristic way: they compare against round-to-nearest, or against a reimplementation of a baseline, and report perplexity on the corpus they tuned on. We set three rules before starting.

1. **Real comparators in the same harness.** `Q3_K_M` and `Q4_K_M` are built by llama.cpp with an importance matrix and evaluated by our code, on the same stack, in the same run. Their bpw is read from the container's actual stored bytes, not from a nominal label.
2. **Same-stack only.** Numbers from different hardware/software stacks are never pooled or compared for a verdict.
3. **Preregistered gates.** PASS(a), the sub-4 claim: a seed variant at <= 3.6 bpw with KL <= KL(`q3km`) and bpw <= 0.92 x bpw(`q3km`). PASS(b), the parity claim: KL <= 1.1 x KL(`q4km`) and bpw <= 0.85 x bpw(`q4km`).

Rule 3 fixed the verdict in advance: it was computable before we knew whether we would like the answer.

1.3 CONTRIBUTIONS

1. A measured ceiling for the seeded-bulk-plus-outlier niche at approximately 3.5 bpw on a 1.7B model (§4), with formal gate failure.
2. The **error-structure** result: a 2.6x behavioural improvement at zero bit cost, measured against a matched-seed-budget control, and *anti-correlated* with plain weight-space error (§5).
3. The **Goodhart geometry** of the weighted-L2 proxy: mechanism dependence of the proxy-to-KL map, quantified as a family displacement rather than an outlier (§6).
4. A **calibration-leakage** measurement on our own protocol (§7).
5. A **scale-inversion** observation: the method degrades from 0.6B to 1.7B while `rel_err` does not (§8).
6. Appendices: a consumer-hardware MoE inference ladder (§A) and a negative depth-routing probe (§B).

---

2. Method

2.1 THE FORMAT

Blocks and basis. A tensor is flattened, zero-padded to a multiple of `C`, and cut into blocks of `C` consecutive weights. Each block is approximated as `w_b ~= U(seed) c`, where `U(seed)` is the `C x P` matrix whose entries are the successive states of a 16-bit Galois LFSR (taps `0xB400`, never allowed to absorb at zero) started at `seed`, mapped affinely into (-1, 1). `U` is a pure function of the seed, so the decoder regenerates it and no part of it is stored.

Fit. Fitting a block is a search over candidate seeds: for each, solve the `P` coefficients in closed form, quantize them, reconstruct, and keep the seed with the smallest block SSE. The candidate set is the deterministic full sweep of all 65,535 non-zero LFSR states when `n_seeds >= 65535` (the budget every headline row in this paper uses) and a `generator_seed`-derived random draw below that; 16,384 is the only smaller budget that appears, and §5.1b is about the difference.

Coefficients. The `P` coefficients are quantized to signed 4-bit integers in [-8, 7] against a shared per-block power-of-two exponent `e = ceil(log2(absmax / 7))`, stored in 4 bits. A block therefore costs 16 bits of seed, 4 of exponent and `4P` of coefficient, and

```
bpw_base = (16 + 4 + 4P) / C
```

is the whole base accounting: no per-tensor scale, no codebook, no index. `c8p3` (`C = 8`, `P = 3`) is `(16 + 4 + 12) / 8` = **4.00 bpw**; `c12p4` (`C = 12`, `P = 4`) is `(16 + 4 + 16) / 12` = **3.00 bpw**.

Side channel. An exactly-stored outlier costs 48 bits: 16 for the fp16 value, 32 for its uint32 flat index. Total cost is the project's only bpw formula, `bpw_base + side_bits / numel`, so a budget of `p` percent adds `0.48 p` bpw. At `p = 1.0` that is 0.48, and `c12p4_awq_p1.0` stores 3.00 + 0.48 = **3.48 bpw**, the figure every seed row in §4 carries.

The W1 objective. The plain fit minimizes `||w_b - U c||^2`; W1 minimizes `||diag(s) (w_b - U c)||^2`, where `s` is the mean |activation| of each input channel, normalized to mean 1 over the tensor, and each of a block's `C` weights takes the `s` of the input channel it belongs to. Both the coefficient solve (weighted normal equations with a 1e-6 relative ridge, in place of the unweighted pseudo-inverse) and the seed argmin score that same weighted SSE, so search and solve optimize one objective. `s` is used at fit time only: the reconstruction is still `U c` with the plain `U`, the stored object is still one seed, one exponent and `P` 4-bit coefficients, and the decoder is untouched. The weighting reshapes the objective while the format stays fixed, which is what licenses the equal-bits comparisons in §5 and §6. W3 is the exception: it folds `s` into the fitted object, so decode needs it back, priced at 16 bits per input channel (`16 / m` bpw).

Formulas, the exact candidate sets and the accounting selftests are in `code/lfsr_core.py`, `code/salience.py` and `code/runner.py`.

Configuration slugs name the geometry: `c12p4` is 12-element blocks with `P = 4` coefficients (nominal base 3.00 bpw); `c8p3` is the wider, cheaper-to-fit 4.00 bpw configuration used in Phase 2.

Suffixes name the *fit mechanism*; the stored format is the same for all of them:

slug	fit objective	coefficient rounding	rotation	calibration
`c12p4`	plain L2	nearest	none	--
`c12p4w`	activation-weighted L2 (W1)	nearest	none	12 prompts
`c12p4wr`	activation-weighted L2	exact search (W2)	none	12 prompts
`c12p4wh`	activation-weighted L2	nearest	Hadamard (W3)	12 prompts
`c12p4w@wt256`	activation-weighted L2	nearest	none	256 wikitext paragraphs

Every `c12p4*` row at a given outlier budget stores exactly the same number of bits. W1, W2 and W3 change *which* coefficients are chosen while their bit cost stays fixed, and the harness asserts this in its accounting selftests. Comparisons across the suffixes are therefore controlled experiments at exactly equal bits, with fit mechanism as the single manipulated variable.

2.2 THE OUTLIER SIDE-CHANNEL

Budget `p` (in percent of weights) selects the entries stored exactly. Three ranking rules were compared in Phase 2: magnitude (`mag`), activation-aware (`awq`, mean |activation| times |weight|), and a spike/low-rank hybrid (`spike`). `awq` won decisively (best KL reduction 64.9% against 34.9% for `mag` and 31.7% for `spike`) and is used throughout Phase 3. Budgets `p` in {0, 0.5, 1.0, 2.0} add {0, 0.24, 0.48, 0.96} bpw respectively.

2.3 EVALUATION PROTOCOL

Teacher-forced mean KL divergence against the bf16 model, 12 prompts, 726 positions, with p95 KL, max KL, and top-1 agreement reported alongside. Controls in every run: `bf16` must give exactly 0.0 KL and reproduce the reference fingerprint; `rtn4` must give KL > 0.

KL is itself a proxy: teacher-forced, 12 prompts, one reference. No downstream-benchmark row exists for any weighted variant. §7 is about the fact that those 12 prompts do double duty.

---

3. Experimental setup

Models: Qwen3-0.6B (Phase 2), Qwen3-1.7B (Phase 3 and 3.6). Hardware: RTX 4090 pods, torch 2.11.0+cu128. Fits are deterministic from `generator_seed = 3407` and cached per tensor.

Stacks are never pooled. The pod's container id is part of its stack string and changes on restart; four container ids appear in the results file, and the cross-stack drift table shows shared rows agreeing bit-identically: evidence the no-pooling rule is being conservative rather than necessary, but it is enforced anyway.

A hardware note. During development, sustained fit runs caused repeated unexplained crashes on one test laptop; rolling the GPU driver back to the vendor-validated version resolved them. All measured results were produced on rented GPUs. Per-tensor caching plus deterministic fits reduced each interrupted run to approximately 20 lost minutes.

---

4. Result 1: the ceiling

All rows same-stack, Qwen3-1.7B:

variant	bpw	mean KL	top-1 agree	note
`q4km` (imatrix)	4.8008	0.032270	96.85%	comparator
`rtn4`	4.5000	0.176448	91.66%	comparator
`q3km` (imatrix)	**3.8478**	**0.204809**	90.29%	**gate target**
`c12p4_awq_p1.0` @16,384 seeds	3.4800	1.818086	66.2%	plain L2, 8.9x off
`c12p4_awq_p1.0` @65,535 seeds	3.4800	1.121121	76.33%	plain L2, W1 control (§5)
`c12p4w_awq_p1.0`	**3.4800**	**0.436118**	84.4%	**W1, best ever, 2.1x off**
`c12p4wr_awq_p1.0`	3.4800	0.485679	--	W2 on top of W1, worse (§6)
`c12p4r_awq_p1.0`	3.4800	0.939671	77.02%	W2 without W1, better (§6.3)
`c12p4w@wt256_awq_p1.0`	3.4800	0.569260	83.45%	recalibrated (§7)
`c12p4w@wt128a_awq_p1.0`	3.4800	0.550612	83.17%	held-out calib draw a (§7)
`c12p4w@wt128b_awq_p1.0`	3.4800	0.656162	82.49%	held-out calib draw b (§7)

Gate verdict: FAIL, `pass_a = False`, `pass_b = False`. No seed variant qualifies under either claim. Nothing in the ladder of attempted improvements changed that, and the ladder is complete.

Read strictly, the 0.436 row is calibrated on its own evaluation prompts (§7); the best row calibrated on held-out text is `c12p4w@wt128a_awq_p1.0` at 0.550612, a 2.7x gap. The true ceiling is therefore worse than the headline.

In summary: at 10% fewer bits than `Q3_K_M`, the method does 2.1x more behavioural damage. That trade favours `Q3_K_M` by a wide margin, and the gap is too large to close by tuning.

---

5. Result 2: the structure of the error (W1)

5.1 THE MEASUREMENT

W1 changes exactly one thing: the fit minimizes an activation-weighted L2 rather than plain L2. The stored format is bit-identical. Both rows below were fitted at 65,535 candidate seeds, so the objective is the only manipulated variable.

Qwen3-1.7B, 3.48 bpw, 65,535 seeds, byte-identical stored format	**plain rel_err**	**act-weighted rel_err**	**mean KL**
`c12p4_awq_p1.0` (plain L2)	0.2192	0.2276	1.121121
`c12p4w_awq_p1.0` (W1)	0.2534	0.1616	0.436118
change	**+15.6% (worse)**	-29.0%	**-61.1% (2.57x better)**

W1's gain came from relocating the error; the total amount of it grew. Under the metric the field routinely uses to rank fits, W1 is the *worse* fit (plain relative weight error rises 15.6%) and it does 2.6x less behavioural damage at exactly the same bits. That is the sharpest form the error-structure result takes anywhere in this project: across fit mechanisms, the sign of the plain proxy inverts.

5.1B DECOMPOSING THE RAW 4.2X DISTANCE

The raw distance from the first plain-L2 row measured (`c12p4_awq_p1.0` at 16,384 seeds, KL 1.818086) to the best weighted row (0.436118) is 4.17x, but that comparison crosses two variables: the unweighted row was fitted at 16,384 candidate seeds and the weighted row at 65,535. The control row above splits it cleanly:

factor	**step**	**KL**	**multiplier**
seed budget	`c12p4` @16,384 -> @65,535	1.818086 -> 1.121121	**1.62x**
fit objective (W1)	`c12p4` -> `c12p4w`, both @65,535	1.121121 -> 0.436118	**2.57x**
(product, the raw distance)		1.818086 -> 0.436118	4.17x

The one-variable W1 number is 2.6x, at zero bit cost, with plain `rel_err` moving the wrong way. The seed-budget factor of 1.62x is itself worth recording: it is close to the 1.8x measured on `c8p3` /1.7B for the same step.

5.2 ELASTICITY: THE PROXY THAT COULD NOT SEE IT

Define elasticity as `d log KL / d log proxy` (how much KL movement accompanied each unit of proxy movement) and compare the W1 step against the same quantity measured inside families where the proxy is known to work (varying only the outlier budget).

step	proxy	elasticity
W1 one-variable control, both @65,535	**plain rel_err**	**-6.5 (wrong sign)**
W1 one-variable control, both @65,535	**act-weighted rel_err**	**2.8**
W1 two-variable comparison (crosses seed budget)	plain rel_err	272.8
W1 two-variable comparison (crosses seed budget)	act-weighted rel_err	3.2
budget sweep, `c12p4@16384`	plain rel_err	23.5
budget sweep, `c12p4@16384`	act-weighted rel_err	6.7
budget sweep, `c12p4w@65535`	plain rel_err	3.8
budget sweep, `c12p4w@65535`	act-weighted rel_err	2.7
budget sweep, `c12p4wr@65535`	plain rel_err	2.3
budget sweep, `c12p4wr@65535`	act-weighted rel_err	1.6
budget sweep, `c8p3@65535`	plain rel_err	15.1

Take plain `rel_err`'s within-family elasticity of 23.5 as its baseline exchange rate. On the one-variable control it predicts the wrong *direction*: `rel_err` rose 15.6%, so the proxy expects KL to rise from 1.121, and KL instead fell to 0.436. Read as an elasticity that is -6.5, against a within-family range of +2.3 to +23.5 for the same quantity. Across fit mechanisms the proxy changes sign, a stronger failure than losing resolution. On the two-variable comparison the same pathology shows up as an absurdly large positive elasticity (272.8) rather than a negative one, which is the weaker version of the same statement.

The weighted proxy, by contrast, moved by a large but entirely *ordinary* amount: -29.0% at elasticity 2.8, squarely inside the 1.6-6.7 range of its own within-family sweeps. W1 moved along the weighted proxy's existing curve. That is the whole empirical case for the weighted objective, and §6 is where it stops.

5.3 THE TWO PROXIES DISAGREE

The control run also supplies two secondary rows: `c12p4_seed_only` @65,535 = 3.450684 (against 7.965954 at 16,384 seeds), and the plain and act-weighted `rel_err` of 0.2192 and 0.2276 used in §5.1. Note that the unweighted fit's act-weighted `rel_err` (0.2276) is *worse* than W1's (0.1616) by 29% while its plain `rel_err` is *better* by 15.6%: the two proxies disagree about which fit is better, which is the cleanest statement of what W1 does.

The methodological point stands on its own: without the matched-budget control, the two-variable comparison would overstate the objective's own contribution by 1.6x. One variable per comparison is what makes the 2.6x attributable.

---

6. Result 3: the Goodhart point of the weighted proxy (W2)

6.1 A STRICTLY BETTER MINIMIZER THAT MAKES THE MODEL WORSE

W2 replaces per-coefficient nearest rounding with an exact search over a 48-candidate exponent-and-rounding grid. The candidate set *contains* the nearest-rounding point, so W2 provably cannot increase the weighted objective on any block; the harness selftests assert this.

outlier budget	wrel `c12p4w` -> `c12p4wr`	delta objective	KL `c12p4w` -> `c12p4wr`	delta KL	elasticity
`seed_only`	0.17035 -> 0.16737	-1.75%	0.545002 -> 0.544878	-0.02%	0.0
`awq_p0.5`	0.16530 -> 0.16251	-1.69%	0.485797 -> 0.489034	+0.67%	-0.4
`awq_p1.0`	0.16161 -> 0.15894	**-1.65%**	0.436118 -> 0.485679	**+11.36%**	**-6.5**
`awq_p2.0`	0.15509 -> 0.15261	-1.60%	0.422626 -> 0.468470	+10.85%	-6.4

Objective lower in 4/4 pairs; KL higher in 3/4. As a sign test on four paired rows that is p = 0.625, which settles nothing by itself. The decisive structure is geometric.

6.2 A DISPLACED FAMILY

Treat the `c12p4w` family as a reference curve (four rows, monotone in both axes), interpolate piecewise-linearly, and ask where each `c12p4wr` row lands on it *at its own weighted error*:

`c12p4wr` row	its wrel	KL the `c12p4w` curve predicts	KL measured	residual
`awq_p2.0`	0.15261	out of range	0.468470	--
`awq_p1.0`	0.15894	0.430593	0.485679	**+0.0551 (+12.8%)**
`awq_p0.5`	0.16251	0.448276	0.489034	**+0.0408 (+9.1%)**
`seed_only`	0.16737	0.510034	0.544878	**+0.0348 (+6.8%)**

All three in-range rows sit **above** the curve, mean displacement **+8.8%** of their own KL. A single outlier is one row off a curve; this is a whole family displaced in one direction, and *inside* itself the `c12p4wr` family is perfectly behaved (Spearman(wrel, KL) = 1.000). It simply lives on a different curve.

The fourth row makes the point without interpolation: `c12p4wr_awq_p2.0` has a **lower weighted error than every `c12p4w` row in existence** (0.15261 against a best of 0.15509) and lands at a KL that two of the four `c12p4w` rows beat. If the objective ranked damage, that would be the best row in the table.

6.3 WHAT THIS MEANS

W1 and W2 are both fit-mechanism changes at byte-identical bits, and both lowered the weighted objective. W1 moved *along* the proxy's curve; W2 moved *off* it, at the wrong sign entirely. Same objective, same units, same model, same bits: one mechanism change was predicted by the proxy and the very next one was not.

The weighted-rel_err-to-KL relation is not a function of weighted rel_err alone; it also depends on how the error was obtained. The proxy has stopped being a proxy and become a coordinate.

Two mechanisms explain the displacement equally well and nothing in the rows above separates them: (i) the weights `s` are noisy, so exact minimization overfits the noise; (ii) weighted L2 genuinely diverges from KL near the optimum. §7 is the cheap experiment that bears on (i).

6.4 ATTRIBUTING THE DISPLACEMENT TO THE METRIC

`c12p4r` applies the identical exact-rounding search **without** W1: plain L2 objective, exact search over the same grid, same 65,535-seed budget. If exact rounding were intrinsically damaging (an overfit search, a pathology of grid-point selection), it should hurt here too. It does the opposite:

pair, 65,535 seeds, byte-identical bits	nearest rounding	exact search (W2)	delta KL
under **plain L2** (`c12p4` -> `c12p4r`), `awq_p1.0`	1.121121	0.939671	**-16.2% (better)**
under **plain L2**, `seed_only`	3.450684	3.231275	**-6.4% (better)**
under **weighted L2** (`c12p4w` -> `c12p4wr`), `awq_p1.0`	0.436118	0.485679	**+11.4% (worse)**
under **weighted L2**, `awq_p2.0`	0.422626	0.468470	+10.9% (worse)

The same search flips sign according to the objective it minimizes: exact coefficient rounding is a good idea under plain L2 and a bad idea under the weighted objective. That rules out the “exact rounding overfits, full stop” reading and localises the Goodhart displacement to the weighted metric itself: it is the weighted objective that has stopped tracking damage near its own optimum, and any mechanism that pushes harder on it inherits the problem.

This strengthens §6.3. It is specifically the *weighted* proxy whose level sets no longer correspond to level sets of KL, while the plain proxy, far from its own optimum and much worse in absolute terms, still behaves monotonically when only the rounding rule changes.

It does not, however, separate mechanisms (i) and (ii) above: noisy `s` and genuine divergence of weighted L2 from KL both predict exactly this pattern, since both are properties of the weighted metric. §7 bears on (i) and is the reason C5 remains open.

---

7. Result 4: we calibrated on our test set

7.1 THE DISCLOSURE

Both the fit weighting `s` and the AWQ outlier ranking are mean |activation| over 12 calibration prompts. **Those 12 prompts are the 12 evaluation prompts.** Every weighted number in this project, including the W1 headline, is calibrated on its own test set. No experiment in the project varied that number until this one.

7.2 THE MEASUREMENT

Activation scales were re-captured from 256 wikitext-2 paragraphs (corpus sha256 `3e4cc617...`) into a separate `act_scales@wt256` file, leaving the 12-prompt scales and every fit depending on them untouched. Recalibration changes no bits.

variant	calib	bpw	mean KL	p95 KL	top-1	act-wtd rel_err
`c12p4w_awq_p1.0`	12 prompts (= eval set)	3.4800	0.436118	--	84.4%	0.1616
`c12p4w@wt256_awq_p1.0`	256 wikitext paragraphs	3.4800	**0.569260**	2.925778	83.45%	0.1656
`c12p4w@wt256_seed_only`	256 wikitext paragraphs	3.0000	0.597257	3.320129	82.08%	0.1748

KL 0.436 -> 0.569, +31%, at identical bits. Scale-drift between the two calibration sets: **cosine median 0.9400**. The recalibrated configuration also fails both gates.

7.3 THE ERROR BAR, AND THE LEAKAGE EFFECT SIZE

A single alternative draw cannot separate a leakage penalty from ordinary calibration-set variance, so we ran two: `wt128a` and `wt128b`, disjoint 128-paragraph halves of the same wikitext-2 corpus, each capturing its own activation scales and each fitted and evaluated independently at 65,535 seeds and identical bits.

variant	calib	bpw	mean KL	p95 KL	top-1	act-wtd rel_err
`c12p4w_awq_p1.0`	12 prompts (= eval set)	3.4800	**0.436118**	--	84.4%	0.1616
`c12p4w@wt128a_awq_p1.0`	wikitext half a	3.4800	0.550612	2.637969	83.17%	0.1664
`c12p4w@wt128b_awq_p1.0`	wikitext half b	3.4800	0.656162	3.337585	82.49%	0.1640
`c12p4w@wt256_awq_p1.0`	both halves (256)	3.4800	0.569260	2.925778	83.45%	0.1656

Two things fall out.

(a) The weighted objective's KL is calibration-sensitive, with a large variance. Two draws from the *same* corpus, differing only in which 128 paragraphs they saw, land 0.106 KL apart: approx. 19% of the lower value. Nothing about the format, the bits, the seed budget or the objective changed between them. **That 0.106 is the noise floor: no comparison between two weighted configurations smaller than approx. 0.1 KL is resolvable in this harness**, which retrospectively disposes of several differences this project treated as meaningful, including the 0.436-vs-0.486 W2 gap in §6 (0.050, well inside the floor; §6's case rests on the four-row family displacement, which is unaffected). The 256-paragraph row at 0.569260 sits inside the a/b bracket, as a pooled draw should.

(b) The leakage advantage has a size. The leaked 0.436118 lies **below the entire held-out band [0.551, 0.656]**, by 0.115 against the nearer edge and 0.220 against the farther one. Calibrating on the evaluation set was worth approximately **0.12 to 0.22 KL**, which is one to two full noise-floor widths and roughly a quarter to a half of the headline number itself. As a ratio, the held-out figure is 1.26x to 1.50x the leaked one.

This completes finding 4. The direction was never in doubt; the magnitude now has a bracket, wide enough that **every weighted number in this paper should be read as a point measurement carrying an approx. 0.1 KL calibration uncertainty**, with the p12-calibrated ones additionally biased low by approximately 0.12 to 0.22.

One quantity is *not* here: the scale-drift cosine median of `wt128a` and `wt128b` against `p12` and against each other was not captured in this run, so the geometric drift measurement exists only for `wt256` (0.9400). The behavioural spread above is the load-bearing number and it does not depend on it.

---

8. Result 5: scale inverts the folklore

model	variant	bpw	mean KL	plain rel_err
Qwen3-0.6B	`c12p4_awq_p1.0` @65,535	3.4800	0.758701	0.2180
Qwen3-1.7B	**`c12p4_awq_p1.0` @65,535**	**3.4800**	**1.121121**	**0.2192**
Qwen3-1.7B	`c12p4_awq_p1.0` @16,384	3.4800	1.818086	0.2547

The first two rows are a matched comparison: same configuration, same outlier budget, same bits, same seed budget, model size the only variable. Behavioural damage rises **1.48x** from 0.6B to 1.7B while per-tensor relative error moves **0.55%** (0.2180 to 0.2192), which is scale-invariance to within measurement grain. Phase 2 on 0.6B had looked encouraging (seeded fits approximately matched RTN4 there, and that is the result that motivated Phase 3). **The method got worse as the model got bigger**, against the usual expectation that larger models carry more redundancy and compress more gracefully.

A comparison against the 16,384-seed 1.7B row would suggest a doubling (0.759 -> 1.818, 2.40x), but those two rows differ in seed budget as well as scale, and the seed-budget difference inflates the number. The one-variable control row (§5) resolves it: the true scale penalty is 1.48x, and the `rel_err` invariance is tight (0.55%).

The operative lesson is the one that generalises: **weight-space error is scale-invariant and behavioural damage is not**, so `rel_err` is at best a prior.

8B. W3: HADAMARD INCOHERENCE IS A DUD, AND WE KNOW WHY

Incoherence processing (rotating the weight matrix by a random Hadamard transform before fitting) is standard practice in the quantization literature and was the obvious third lever. Measured on 1.7B, **W3 is 29% worse than W1** on the shared metric.

The mechanism is understood: Qwen3 rows are already near-Gaussian (kurtosis approx. 3.9), so there is no incoherence left to fix, and the rotation *destroys* the per-block concentration that lets four coefficients specialise.

---

9. Discussion

9.1 WHAT A BETTER OBJECTIVE WOULD HAVE TO DO

In falsifiable form, from the full analysis in `docs/experiment/PROXY-ALIGNMENT.md`:

- **C1.** Within a fixed fit mechanism, both proxies rank KL correctly. *Six families, two models, Spearman +1.000, no counterexample.*
- **C2.** Plain per-tensor `rel_err` cannot compare fit mechanisms. *Strengthened by the one-variable control: across the W1 step the proxy points the wrong way (15.6% worse fit, 2.6x less damage).*
- **C3.** Lowering the current weighted L2 further does not lower KL. *Measured at the frontier by a mechanism that provably lowers the objective and changes nothing else, and confirmed to be a property of the weighted objective rather than of the mechanism, since the same mechanism under plain L2 lowers KL 16% (§6.4).*
- **C4.** A better objective must be **mechanism-invariant**: fits reached by different mechanisms at equal objective value must land at equal KL. *Nearest-vs-exact rounding is already such a test pair, and the current objective fails it by 7-13% of KL while plain L2 passes it in sign. Any candidate objective should be run through this test before it is run through a gate.*
- **C5.** The objective weights are a measurement whose error is now quantified but not decomposed. *Two disjoint held-out draws span 0.106 KL (§7.3); what remains unknown is how much of that is estimator noise in `s` versus real corpus difference, which is the question that separates mechanisms (i) and (ii) in §6.3.*

9.2 WHY NOT ANOTHER CYCLE

The remaining ideas (GPTQ-style error compensation, joint seed-and-rounding search, richer calibration) are each a new spec-build-measure cycle aimed at a measured 2.1x gap, using a proxy that is measured to have exhausted its alignment with KL. More optimization pressure against an exhausted proxy is how §6 happened. The prerequisite is a mechanism-invariant objective (C4), which is a research project in its own right.

9.3 LIMITS OF THIS WORK

- **n is small and structured.** The seed rows sit in a handful of families, most of four rows; within a family the four rows are one fit plus three refits that reuse it, so a Spearman of +1 is worth far less than +1 on four independent samples, and its exact two-sided p cannot go below 0.083. The `wt128a` and `wt128b` families carry two rows each and support no within-family rank statistic at all; they exist to bracket a single number (§7.3), not to rank.
- **The calibration error bar rests on n = 2.** Two draws give a spread, not a distribution. The 0.106 KL figure is the observed distance between two samples and should be treated as an order-of-magnitude noise floor, not a standard deviation.
- **One model carries the entire objective comparison.** Every weighted row in existence is Qwen3-1.7B; the 0.6B twin was never re-run.
- **KL is a proxy too**: teacher-forced, 12 prompts, 726 positions. No downstream-benchmark row exists for any weighted variant.
- **Back-filled weighted proxies** for the unweighted 1.7B rows were recomputed locally from 80-81 of 196 cached tensors, not measured on the pod. The subsample plain rel_err tracks the full row to 3-4 decimals, which is the only representativeness evidence offered.
- **A partial-fit trap exists in the raw results file.** Two `--layers-limit` smoke rows carry full-model-shaped bpw and KL and, taken at face value, produce a spectacular *fake* Goodhart result. They are excluded here and detectable from their own effective-bytes arithmetic (`flag_partial_fits` in `code/proxy_alignment.py`).

- **No causal claim** is made about why the §6 displacement exists.

Appendix A: Consumer-hardware MoE inference ladder

A separate line of work asked what a 30B-class mixture-of-experts model actually does on an 8 GB consumer laptop GPU, framed as a bytes-touched-per-token roofline: `decode_tok_s ~= B_eff / bytes_per_token`. Hardware: RTX 5070 8 GB laptop, 24 logical CPUs, Windows 11.

> `-ncmoe` (`--n-cpu-moe`) is llama.cpp's own existing feature. This work measured it, found the optimal operating point for this hardware, and shows how others can tune it on hardware smaller than the full model would normally need. The contribution here is the offload curve, the location of the cliff, and the exact bytes-per-token accounting that explains both.

Qwen3-30B-A3B, 1,919.6 MB touched per token (exact GGUF tensor accounting; routed-expert stacks scaled by 8/128 experts used):

`-ncmoe`	prefill tok/s	decode tok/s	VRAM peak
48 (all routed experts on CPU)	348.7	**38.21**	1,386 MiB
40	470.1	45.97	4,324 MiB
32	544.9	52.47	7,060 MiB
30	578.0	**54.19**	7,708 MiB
28	113.5	**10.50**	7,874 MiB

The curve rises smoothly to 54.2 tok/s and then **collapses 5.2x at `-ncmoe 28`** as the working set stops fitting in 8 GB. Server-mode real throughput at `-ncmoe 32` is **48.77 tok/s**. Against the stated win condition of 12 tok/s, every rung from 48 down to 30 passes and 28 fails.

Note that rows with experts on CPU report a *blended* implied bandwidth: weights span GDDR7 (attention and KV on GPU) and DDR5 (routed experts on CPU), so those numbers are not comparable to the pure-VRAM rungs.

Speculative decoding is negative on this hardware:

variant	draft	accept rate	decode tok/s	speedup
Qwen3-4B server-spec	Qwen3-0.6B	47.4%	67.52	**0.87x**
Qwen3-4B server-spec	Qwen3-0.6B	51.4%	55.08	**0.71x**
Qwen3-30B-A3B server-spec	n-gram	0%	48.10	**0.99x**

At batch 1 on a bandwidth-starved machine the draft model's own weight reads cost more than verification amortizes. Reporting this matters: speculation is widely assumed to be free upside.

Bug encountered: llama.cpp b10502 crashes with *"invalid vector subscript"* when a dense draft model is combined with `-ncmoe`.

Full rows, the blended-bandwidth caveat, and the bytes-per-token methodology with its six stated biases: `ladder/results.md`.

Appendix B: Skip-depth probe, negative, with one nugget

A depth-routing probe (skip/keep/repeat over decoder layers, evaluated by the same KL protocol) came back **negative**: there is no broad layer slack to harvest on this model. The single stacking-worthy result is **layer 27: KL 0.145 at 2.9% of bytes touched.**

> **Footnote required whenever that 0.145 is quoted against `q3km` 0.205, `rtn4` 0.176, or the 0.436 headline:** the skip probe evaluated **731 positions** against the seed harness's **726** (container drift). It is not a like-for-like KL. Cross-quotes are indicative only.

A build-time audit of the reference implementation (PoLar, ICML 2026, arXiv:2606.06574) found material divergences between the released code and the paper: the headline accuracy protocol is best-of-5 scored against ground truth with a **default-on short-circuit that awards credit by matching the sample's own answer-key file without running the model**; the MCTS supervision generator that produces its only training signal is not in the release; and the "lightweight 2M-parameter predictor" requires a frozen **600M** sentence encoder resident at inference, byte-negative for a bandwidth argument at 1.7B scale. Full audit: `skipdepth/NOTES-polar.md`.

Appendix C: Reproduction

Harness, every result row, and all run scripts are in this repository. Fits are deterministic from `generator_seed = 3407` and cached per tensor. Model weights, fit caches, and reference logits are excluded as large and fully regenerable. See the repository README for command lines.

Appendix D: Operational lessons

The four that generalise beyond this project:

1. **One variable per probe.** W1 changed only the fit objective; that is the entire reason its result is attributable. §5.1b shows what happens when a comparison crosses one extra variable: a multiplier overstated by 1.6x until the matching control row exists.
2. **Measure cheaply first.** A single-tensor CPU pre-measurement predicted the W3 result before any GPU run was committed.
3. **Negative results need comparators as much as positive ones.** Real `Q3_K_M` / `Q4_K_M` anchors turned "it feels bad" into "8.9x off, then 2.1x off", which is what made the W1 win legible in the first place.
4. **Silent failure is the enemy.** Assert artifacts exist before declaring a stage passed; `set +e` wrappers and shell pipelines eat failures and will happily keep running a job that died an hour ago.